MEGATHON X — M1 APPROACH

Team Number: _____
Project: ShopSaarthi AI – An Accessible, Multilingual AI-Powered E-Commerce Assistant
Theme: Accessible AI for All

1. Problem Statement and Understanding

- Conventional e-commerce depends on visually dense screens, small text, complex descriptions, typing, and English-first interaction.
- **Visual impairment:** Product discovery needs screen-reader-compatible structure, meaningful descriptions, keyboard access, and spoken information.
- **Hearing impairment:** Audio-only information excludes users without captions, readable text alternatives, and visible confirmations.
- **Hand-motor disability:** Small controls, complex navigation, and extensive typing impede browsing and checkout.
- **Language barrier:** Users who prefer Telugu or Hindi may struggle to understand products and make informed choices.
- **Objective:** Enable more independent product discovery and informed shopping through accessible interaction and supported languages.

2. Idea / Abstract

ShopSaarthi AI is a proposed accessibility-first, multilingual e-commerce web application. It combines a structured catalogue with AI-assisted product discovery and understandable product information. Users will search through natural language or, where supported, voice in **English, Telugu, and Hindi**. The planned Bhashini speech workflow is **ASR → translation/language processing → AI response → response translation → TTS**; text interaction and user-selected accessibility/language preferences remain available.

- **Product understanding:** Simplify specifications, summarize reviews, and compare products using actual catalogue price, ratings, features, and specifications.
- **Product discovery:** Provide catalogue-grounded recommendations and budget filtering.
- **Accessible design:** Screen-reader-friendly navigation, meaningful descriptions, keyboard controls, and TTS (visual); captions, text alternatives, and visible confirmations (hearing); large, spaced controls, simplified navigation, and reduced typing (motor).
- **Reliability:** If AI or speech services fail, provide a clear message and retain text search and catalogue browsing. Cart and simulated checkout are included only if implemented.
- **Innovation and impact:** Unite multilingual speech, accessible interaction, and catalogue-grounded shopping support to help users make more informed, independent choices. These capabilities are planned; no unverified integration or results are claimed.

3. High-Level Approach, Pipeline, and Architecture

```
User (speech/text; selected language/accessibility preferences)
  ↕
Accessible React + Vite frontend
  ↕ requests and accessible results
Python + FastAPI backend
  ├─ JSON product catalogue (SQLite optional)
  ├─ LLM API: grounded explanations, summaries, comparisons
  └─ Bhashini: ASR, translation/language processing, TTS
  ↓
Validated catalogue-grounded response → text/captions/speech as supported
```

- **Flow:** Frontend sends a query to FastAPI; backend retrieves catalogue records and provides available facts to the LLM. Bhashini processes supported speech/language steps. The backend returns results for accessible presentation. Input validation, clear errors, and text fallback address service failures. No automatic disability detection or offline operation is assumed.
- **Proposed stack:** React/Vite; Python/FastAPI; JSON (SQLite optional); LLM API such as Gemini if selected, available, and permitted; Bhashini; GitHub; suitable frontend/backend hosting.

4. Planned Testing Methodology

- **Functional:** Search, budget filter, product details, comparison, and simulated checkout if implemented.
- **Accessibility:** Screen-reader compatibility, keyboard navigation, text alternatives/captions, visible confirmations, and large controls.
- **Language/speech:** English, Telugu, Hindi; Bhashini recognition, translation, and synthesis, including unclear speech and service failures.
- **AI grounding:** Check explanations, summaries, and comparisons against catalogue records; ensure absent facts are not invented.

- **Reliability/integration:** Test empty results, invalid product IDs, unavailable APIs, network failures, and end-to-end frontend/backend/LLM/Bhashini flows.
- Record actual pass/fail results after testing; this document does not assert results.